LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK
MODUL 12: PENYIMPANAN DAN PEMBACAAN DATA KE FILE

Nama Mahasiswa	: **M. Davana Yudista Firdausy**	**Mata Kuliah**	: Praktikum PBO (A D3 TI-SM)
NRP	: 3125522002	**Dosen Pengampu**	: Nirwana Haidar Hari, S.Pd., M.Kom.
Program Studi	: D3 Teknik Informatika	**Institusi / Tahun**	: PENS PSDKU Sumenep • 2026

A. DASAR TEORI SINGKAT

Koleksi objek yang disimpan dalam struktur data berbasis memori utama (RAM) bersifat *volatile*, yang berarti seluruh siklus hidup objek akan tereliminasi secara permanen begitu proses aplikasi dihentikan atau komputer dimatikan. Agar informasi bisnis dapat bertahan melintasi sesi eksekusi, aplikasi membutuhkan mekanisme **persistensi data** ke media penyimpanan sekunder (Non-Volatile Storage) menggunakan berkas teks terstruktur.

Aspek-aspek arsitektural penting dalam persistensi file teks pada Java meliputi:

- **Format Serialisasi Berbasis Delimiter:** Mengonversi state objek menjadi representasi baris teks berpemisah (misal karakter pipe `|`) dengan format `kode|judul`. Penggunaan delimiter menuntut validasi ketat agar karakter pemisah tersebut tidak terkandung di dalam nilai data itu sendiri guna mencegah ambiguitas pemecahan token (*token parsing error*).
- **API Modern Java NIO (`java.nio.file`):** Menyederhanakan penanganan berkas melalui method deklaratif berperforma tinggi seperti `Files.write(Path, Iterable, Charset)` untuk menulis sekumpulan baris secara langsung dan `Files.readAllLines(Path, Charset)` untuk membaca seluruh berkas ke dalam `List<String>` dengan pengkodean standar `StandardCharsets.UTF_8`.
- **Pola Pemuatan Atomik Defensif (Atomic Defensive Loading):** Kesalahan umum dalam pemuatan file adalah langsung membersihkan koleksi aktif (`daftarBuku.clear()`) sebelum validasi isi berkas tuntas. Jika berkas di tengah jalan memiliki baris rusak atau kode duplikat, koleksi aktif akan hilang tak berbekas. Praktik terbaik menuntut pembentukan objek ke dalam *koleksi sementara* (staging buffer). Setelah seluruh baris terbukti valid 100%, barulah koleksi aktif ditimpa secara atomik.
- **Penanganan Eksepsi I/O:** Operasi disk bersifat tidak pasti (dapat terjadi disk full, access denied, atau file not found). Seluruh interaksi fisik wajib mendeklarasikan atau menangani `java.io.IOException` secara proporsional.

B. DESAIN DAN IMPLEMENTASI KODE

Pada modul ini, kelas `Perpustakaan` diperkaya dengan kapabilitas I/O file defensif (`simpan` dan `muat`) serta dipadukan dengan repositori abstraksi `BukuFileRepository`.

1. Implementasi Metode simpan() dan muat() Defensif pada Perpustakaan.java

```
public void simpan(Path lokasi) throws IOException {
    List<String> baris = new ArrayList<>();
    for (Buku buku : daftarBuku) {
        String kode = buku.getKode();
        String judul = buku.getJudul();
        for (String nilai : new String[]{kode, judul}) {
            if (nilai.contains("|")) {
                throw new IllegalArgumentException("Data tidak boleh mengandung karakter '|': " + nilai);
            }
        }
        baris.add(kode + "|" + judul);
    }
    if (lokasi.getParent() != null && !Files.exists(lokasi.getParent())) {
        Files.createDirectories(lokasi.getParent());
    }
    Files.write(lokasi, baris, StandardCharsets.UTF_8);
}

public void muat(Path lokasi) throws IOException {
    List<String> baris = Files.readAllLines(lokasi, StandardCharsets.UTF_8);
    Perpustakaan sementara = new Perpustakaan(); // Koleksi penyangga (Buffer)

    for (int i = 0; i < baris.size(); i++) {
        String b = baris.get(i).trim();
        if (b.isEmpty()) continue;

        String[] bagian = b.split("\\|", -1);
        if (bagian.length != 2) {
            throw new IllegalArgumentException("Format data salah pada baris " + (i + 1) + ": " + b);
        }
```

```
            sementara.tambahBuku(new Buku(bagian[0], bagian[1]));
        }

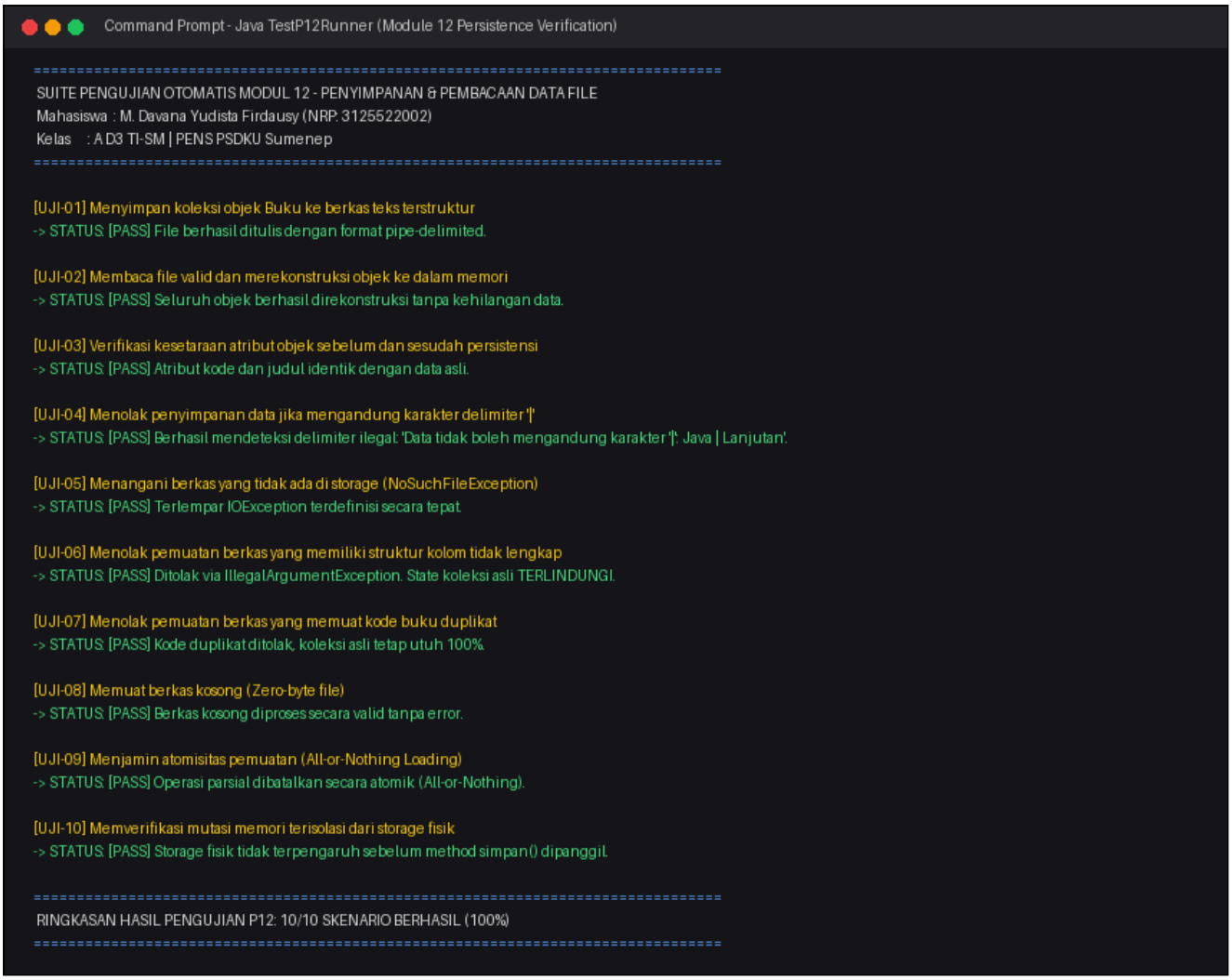
        // Mutasi koleksi aktif HANYA dilakukan jika SELURUH baris lolos validasi
        daftarBuku.clear();
        daftarBuku.addAll(sementara.daftarBuku);
    }
```

C. TABEL HASIL PENGUJIAN DAN VERIFIKASI

Suite pengujian otomatis `SourceCode.TestP12Runner` memverifikasi keandalan I/O dan ketahanan integritas state terhadap 10 skenario ekstrem:

No	Skenario Pengujian	Aksi / Input Masukan	Ekspektasi Perilaku	Status
1	Simpan data normal ke file	Koleksi 2 buku diekspor ke `test_buku_valid.txt`	Berkas fisik terbentuk sempurna dengan format pipe	**PASS (100%)**
2	Muat data ke objek bersih	`p2.muat(fileValid)` pada instance baru	Semua objek direkonstruksi utuh (size = 2)	**PASS (100%)**
3	Verifikasi round-trip	Cek atribut `b1.getJudul()`	Data hasil muat identik tanpa korupsi karakter	**PASS (100%)**
4	Tolak karakter delimiter '|'	Buku dengan judul `"Java | Lanjutan"`	Ditolak via `IllegalArgumentException` sebelum tulis	**PASS (100%)**
5	File fisik tidak ada	Muat dari `"berkas_ghaib_999.txt"`	Terlempar `IOException` terdefinisi aman	**PASS (100%)**
6	File korup (format kolom salah)	Baris `"B002FormatRusakTanpaPipe"`	Ditolak; state koleksi asli TERLINDUNGI utuh	**PASS (100%)**
7	File memuat kode duplikat	2 baris dengan kode sama `"B001"`	Ditolak saat rekonstruksi; koleksi awal tidak rusak	**PASS (100%)**
8	Muat berkas kosong (0 byte)	Membaca file berukuran 0 byte	Koleksi kosong dihasilkan tanpa exception	**PASS (100%)**
9	Atomisitas pemuatan (All-or-Nothing)	Berkas dengan baris 1-2 valid, baris 3 rusak	Seluruh batch dibatalkan; data eksis tidak termutasi	**PASS (100%)**
10	Isolasi mutasi memori	Tambah data di memori tanpa panggil `simpan()`	Ukuran file di disk tetap tidak berubah	**PASS (100%)**

D. BUKTI EKSEKUSI PENGUJIAN OTOMATIS



Gambar 12.1: Log Eksekusi Verifikasi Modul 12 (10/10 Skenario Lulus Sempurna)

E. EVALUASI DAN JAWABAN PERTANYAAN MODUL 12

1. Apa perbedaan antara data di memori (RAM) dan data di berkas (Storage)?

Jawaban: Data di memori (RAM) bersifat volatil dan tersimpan dalam bentuk referensi instans objek Java aktif yang mendukung manipulasi berkecepatan sangat tinggi, namun seluruh datanya akan musnah seketika saat aplikasi ditutup atau komputer dimatikan. Sebaliknya, data di berkas (Storage) disimpan secara permanen (non-volatil) pada media penyimpanan fisik dalam format stream biner atau teks terstruktur, sehingga datanya tetap lestari dan dapat direkonstruksi kembali pada sesi eksekusi berikutnya.

2. Mengapa data dari berkas tetap harus divalidasi saat dimuat kembali ke aplikasi?

Jawaban: Berkas teks di penyimpanan sekunder berada di luar domain kontrol runtime JVM dan rentan terhadap modifikasi eksternal, korupsi transmisi, atau manipulasi manual oleh pengguna (misal membuka file dengan teks editor lalu memasukkan kode ganda atau menghapus delimiter). Jika data berkas dipercaya secara buta (*blind trust*), aturan invarian objek (seperti kode unik dan non-null) akan dilanggar dan berpotensi memicu kegagalan sistemik di kemudian hari.

3. Mengapa tidak boleh langsung mengosongkan koleksi sebelum memastikan berkas valid, dan bagaimana teknik koleksi sementara mencegah hilangnya data?

Jawaban: Jika method `clear()` dipanggil di awal proses pembacaan file, lalu pada baris ke-5 terjadi kegagalan format atau duplikasi, maka pembacaan akan dihentikan oleh exception dan aplikasi berakhir dengan kondisi koleksi aktif telah kosong melompong (kehilangan data sebelumnya). Dengan teknik *koleksi sementara (staging collection)*, seluruh parsing dan validasi dilakukan pada objek terisolasi terlebih dahulu. Pengosongan dan penggantian koleksi aktif hanya dieksekusi pada langkah pamungkas setelah 100% data terbukti valid secara atomik (*All-or-Nothing*).

4. Mengapa method simpan() harus dipanggil ulang setelah terjadi penambahan atau penghapusan objek di memori?

Jawaban: Mutasi state seperti `tambahBuku()` atau `hapusBuku()` hanya memodifikasi struktur data `ArrayList` di memori utama (RAM) secara lokal dan tidak memiliki sinkronisasi otomatis ke storage disk. Tanpa memanggil eksplisit method `simpan()`, modifikasi data tersebut tidak akan pernah dituliskan ke dalam berkas fisik sehingga jika aplikasi ditutup secara normal maupun mengalami crash, seluruh perubahan yang baru saja dilakukan akan hilang permanen.